Now, 3 is lost Hence RTO occures and send whole buff again (3,4,5,6)

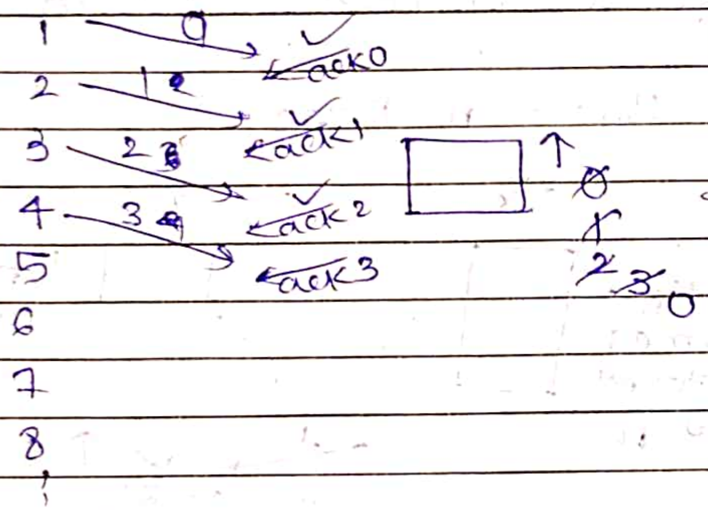
- Here cumulative ack. we deal with.

⟹ outstanding = How many frames can be send at a time. and wait for acknowledgement

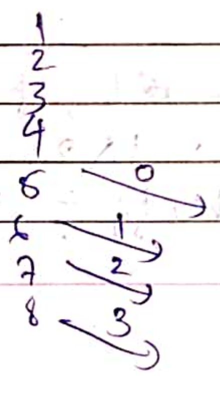
→ 0,1,2,3,0,1,2,3.

- If I want to have 2 bit seq. no. ?
- In this case 4 frames can't be outstand.

Burst error
- For 1 sec there is error

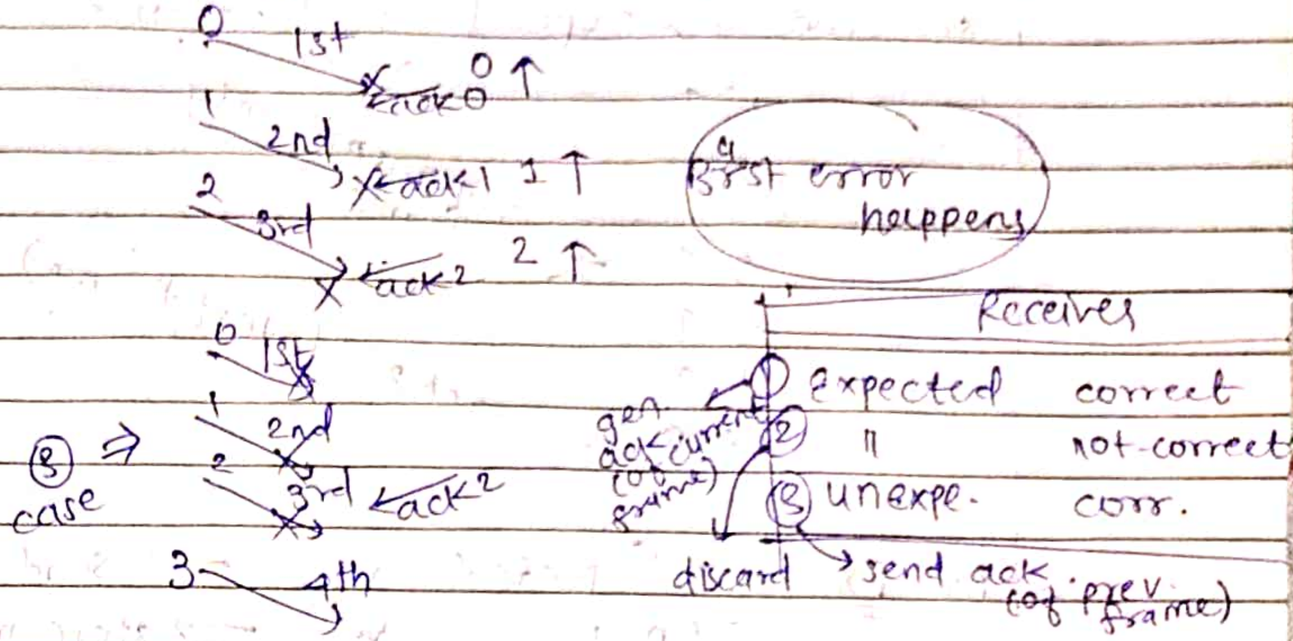


If Burst error happens, all ack. are lost
At tx. timeout at frame 1, and all frames are retransmitted.



← same frame will be sent (No reliability)

- Hence fig for 3 frame.



→ In general n bits ~~frame~~ (seq. no.) then $2^n - 1$ frames can be outstanding.

→ Better than stop & wait.

→ limitation : Only one buff. at Rx.

- Selective Repeat Protocol (TCP uses this)

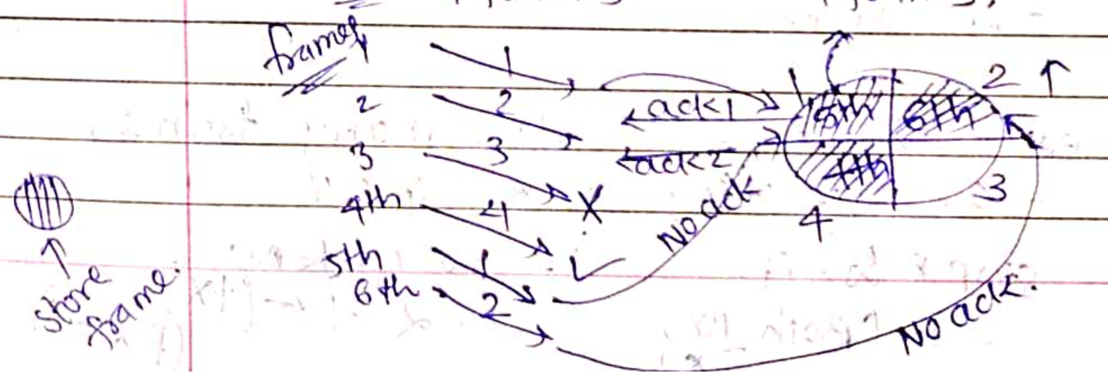
RTO
= 2 * RTT

(Round Trip time)

Tx ⟷ Rx

Both side buff. size is same.

ex 4 frames. 4 frames.



3 is lost = discard.

⇓

RX waits for 3

whichever frame is facing timeout selectively send repeat that frame

1 ↑
2 ↑
3 ↑ → retransmit only frame 3.
4 ↑
1 ↑
2 ↑ ← ack2 (latest frame)

At RX

Expected ✓ → send ack.
Un " ✓ → store in buff

Incorrect → Discard.

→ If in betⁿ one ack is lost
↳ No trouble Because we deal with cumulative ack. next ack. will take care of prev. ack

→ If one frame is dropped. ?
↳ timeout for that frame. below that, frames are stored in buff and once frame which faced timeout is entertain then all other frames are sent.

→ Sender's window = How many frames are sending at a time ?

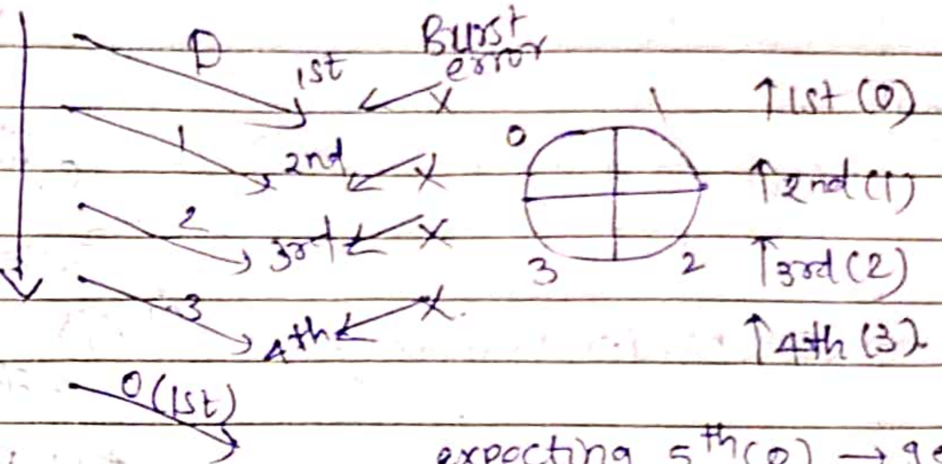
Stop & Wait = 1. (Both RX, TX)

Go Back N $= 2^n - 1$ ← (TX) (RX = 1)

⇒ 2 bit seq number → How many frames can be outstanding.

or seq no. is 2 bit long → what should be sender's window?

All ack lost, but all 4 frames are received. so, receiver expecting 5th frame but 1st is resend

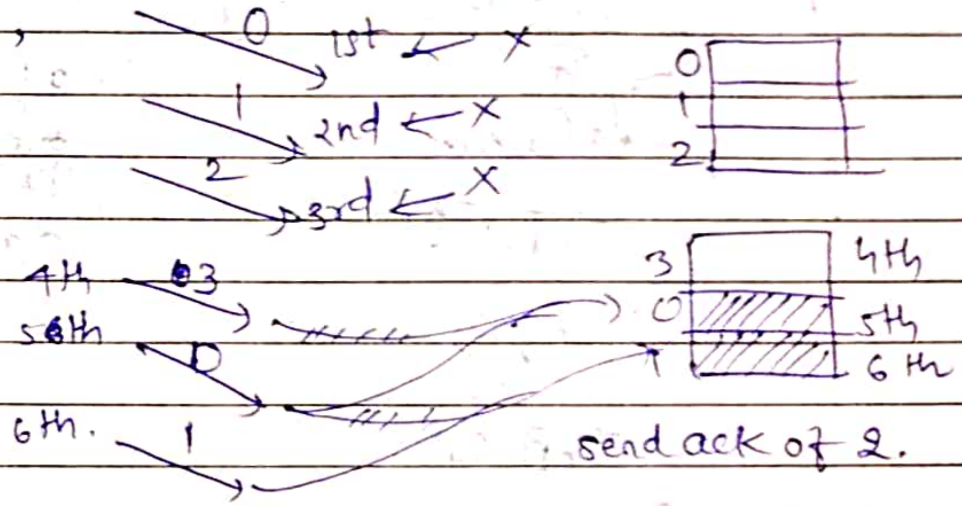


expecting 5^{th}(0) → getting 1st (0)

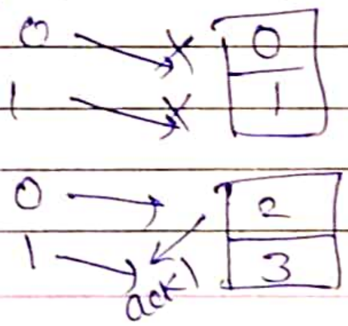
(giving same frame.)

Hence 4 is not answer.

try for 3,



try for 2, (work well)



Sender's window $= 2^{n-1}$

receiver's window $= 2^{n-1}$

DATE
PAGE

→ <u>Transreceiver:</u> Transmitter + Receiver

A B

(0, A_0) →

← ack0 — Combine ack & data ⇓ process is known as Piggy backing (Bcoz header & taier are also same)

← (0, B0)

ack0 →

(1, A_1) →

← ack1

← (1, B1)

frame structure (seq no, data)

↓

send whole rather than seperate ⟹ (seq no, Ack no, data)

(0, —, A_0) →

Ⓐ ← (0, 0, B0) Ⓑ

(1, 0, A_1) →

← (1, 1, B1)

(0, 1, A_2) →

← (0, 0, B2)

(1, 0, A_3) →

← (1, 1, B3)

If in bet^n if ~~we~~ TX don't have data then

send ack | wait some time (much smaller than RTO)

⇑

known as auxiliary timer or ack. timer